Deliverable #5: Service-Oriented Software Engineering

Part 1: Service Engineering Application

1. Service Candidate Identification

Based on the principles of Service-Oriented Architecture, the following services were identified for the system:

- Item Management Service
    - Classification: Business Service.
    - Justification: This service manages the core entity of the application (lost items). It is highly reusable and independent, as it can be utilized by students searching for lost property, or by security officers managing inventory.
- Claim Management Service
    - Classification: Business Service.
    - Justification: This service handles the lifecycle of item claims. It operates independently to track which student is claiming which item, allowing for clear separation of concerns.
- Claim Resolution Workflow
    - Classification: Coordination Service (Composite).
    - Justification: This service orchestrates the Item and Claim business services to execute a complete real-world process without requiring the client to make multiple manual API calls.

2. Service Requirements Specification

For the Item and Claim Business Services:

- Functional Requirements: The system must allow authorized users to Create, Read, Update, and Delete (CRUD) records in the SQL Server database.
- Non-Functional Requirements: The API must be stateless, respond within 500ms, and ensure data integrity between parent and child tables.
- Inputs/Outputs & Constraints: The services must accept and return data exclusively in application/json format. The system must enforce constraints, such as ensuring an UploadedByUserID is present when creating an item.

3. Service Interface Design

The APIs were designed following RESTful principles :

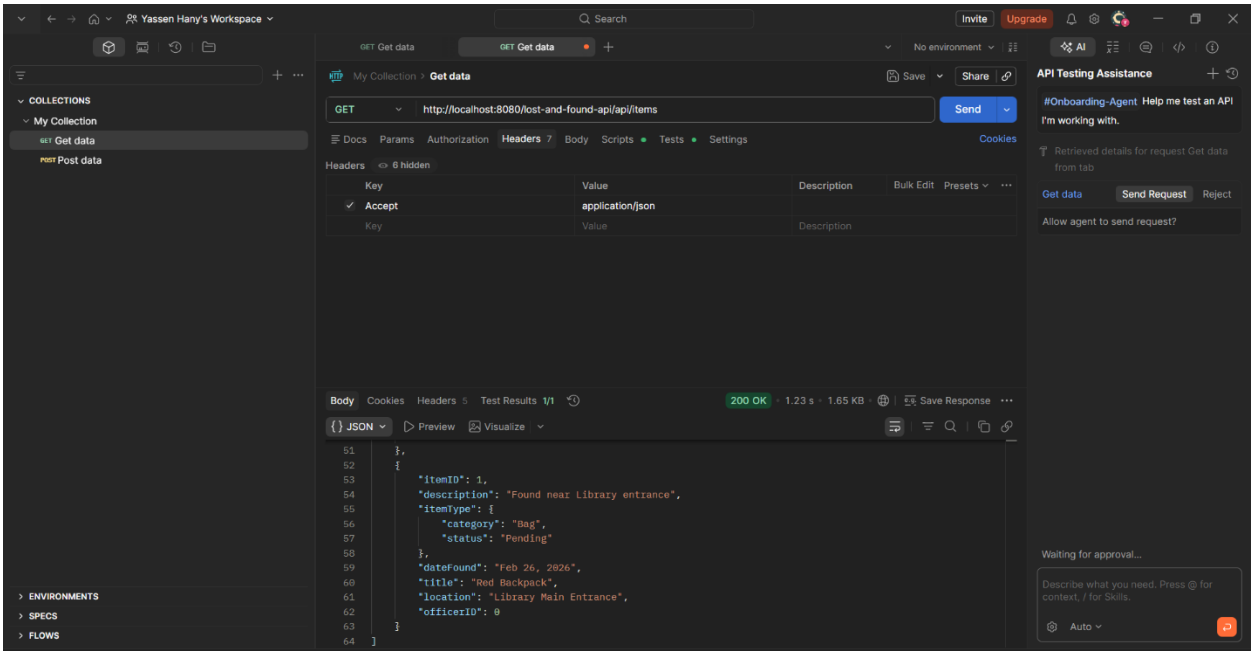
- **Resources & Endpoints:** /api/items and /api/claims.
- **HTTP Methods:** GET (Retrieve), POST (Create), PUT (Update), and DELETE (Remove).
- **Format:** JSON Request and Response payloads.

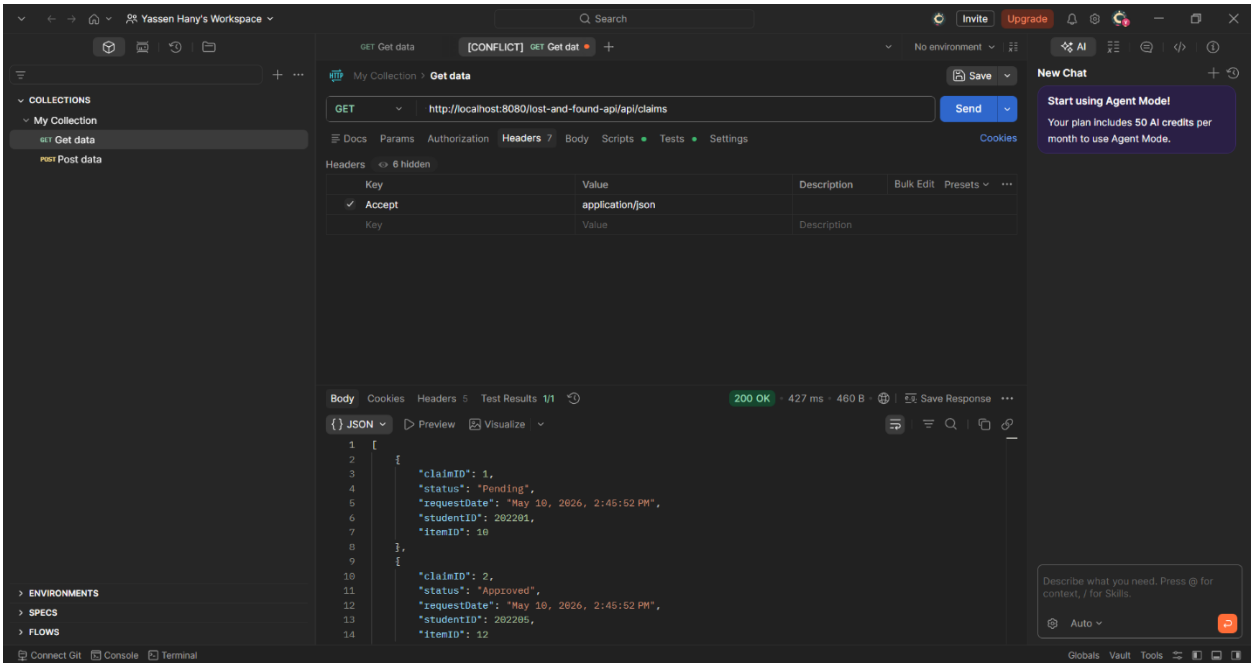
Part 2: REST API Implementation & Testing

The following screenshots demonstrate successful Postman tests for the implemented REST APIs.

1. GET APIs (Retrieve Data)

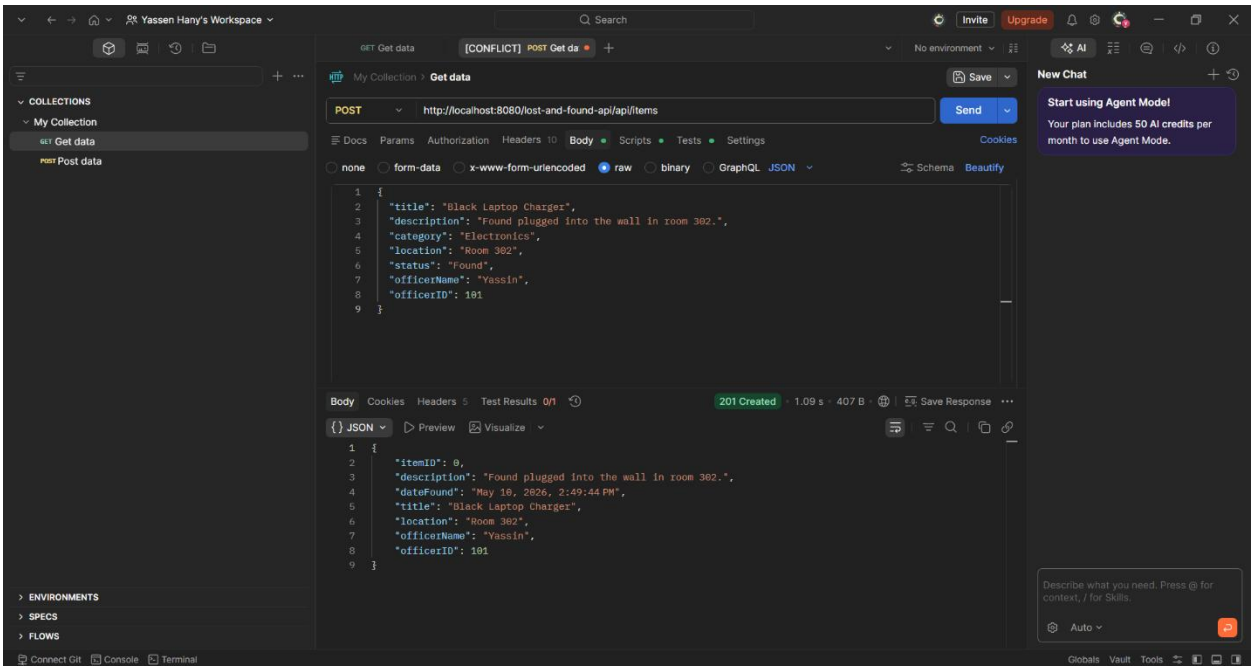
Endpoint: GET /api/items & GET /api/claims

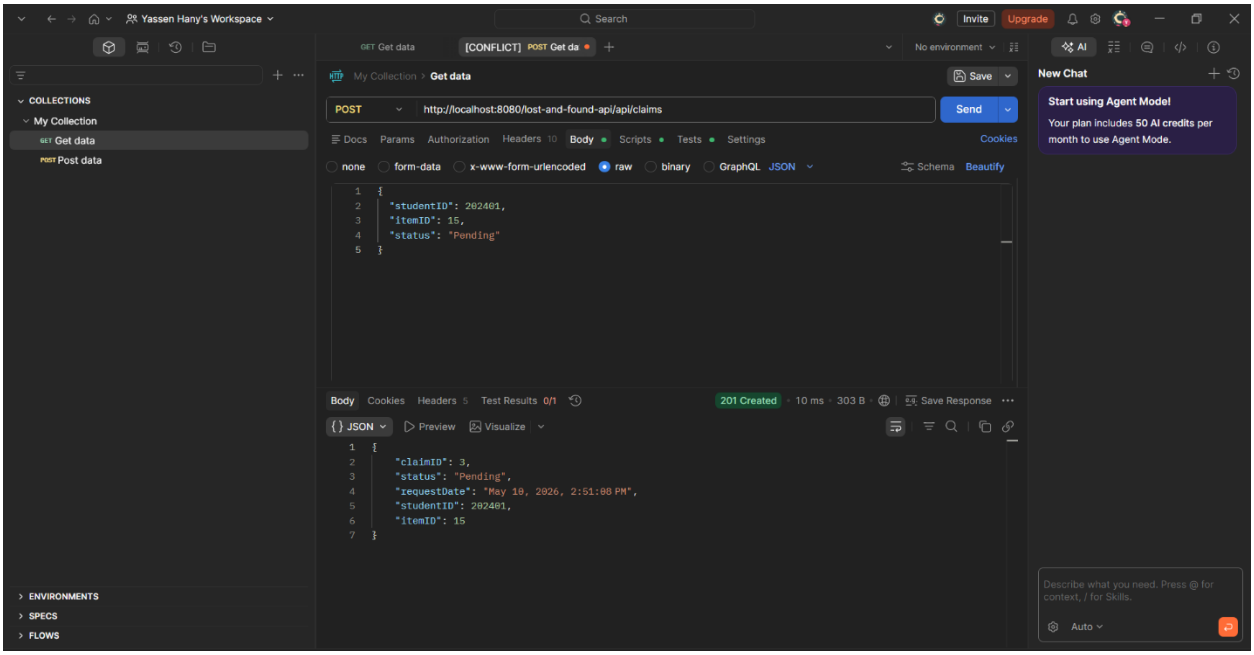




2. POST APIs (Create Data)

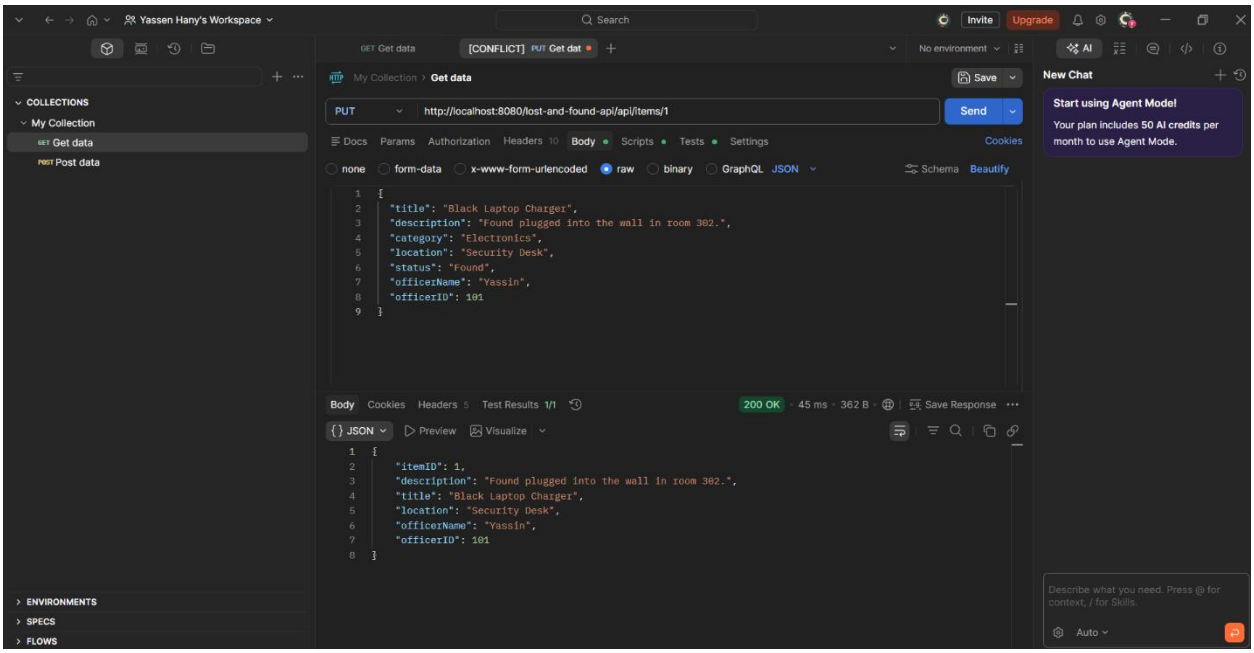
Endpoint: POST /api/items & POST /api/claims

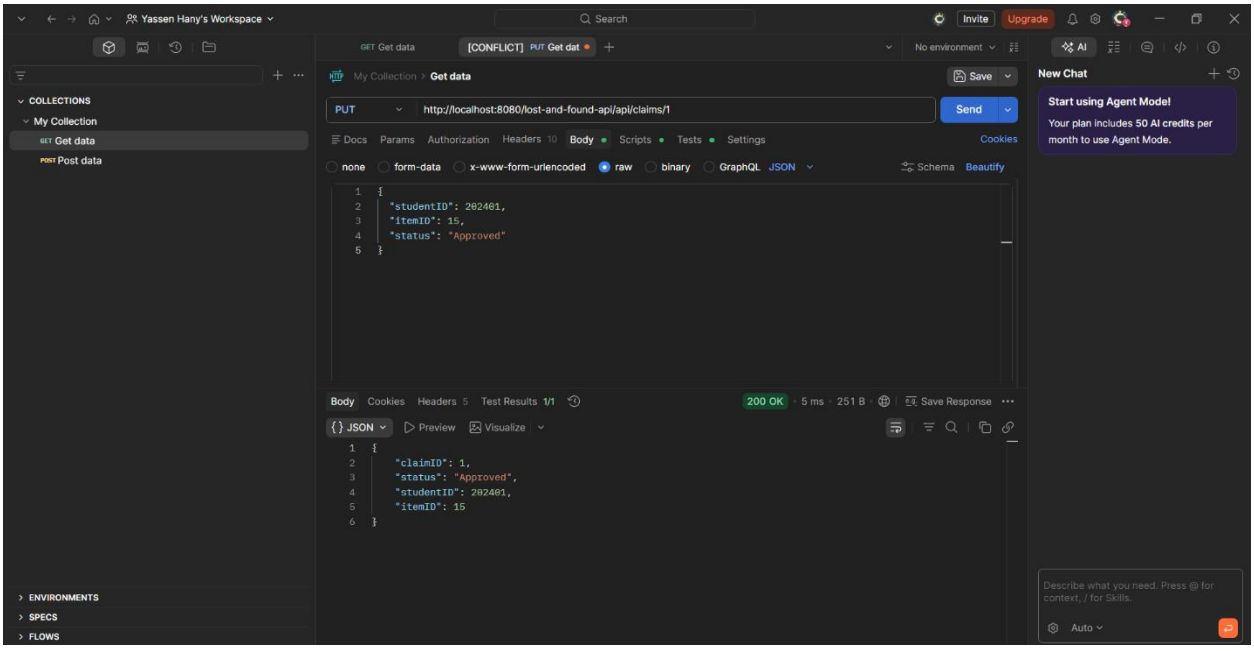




3. PUT APIs (Update Data)

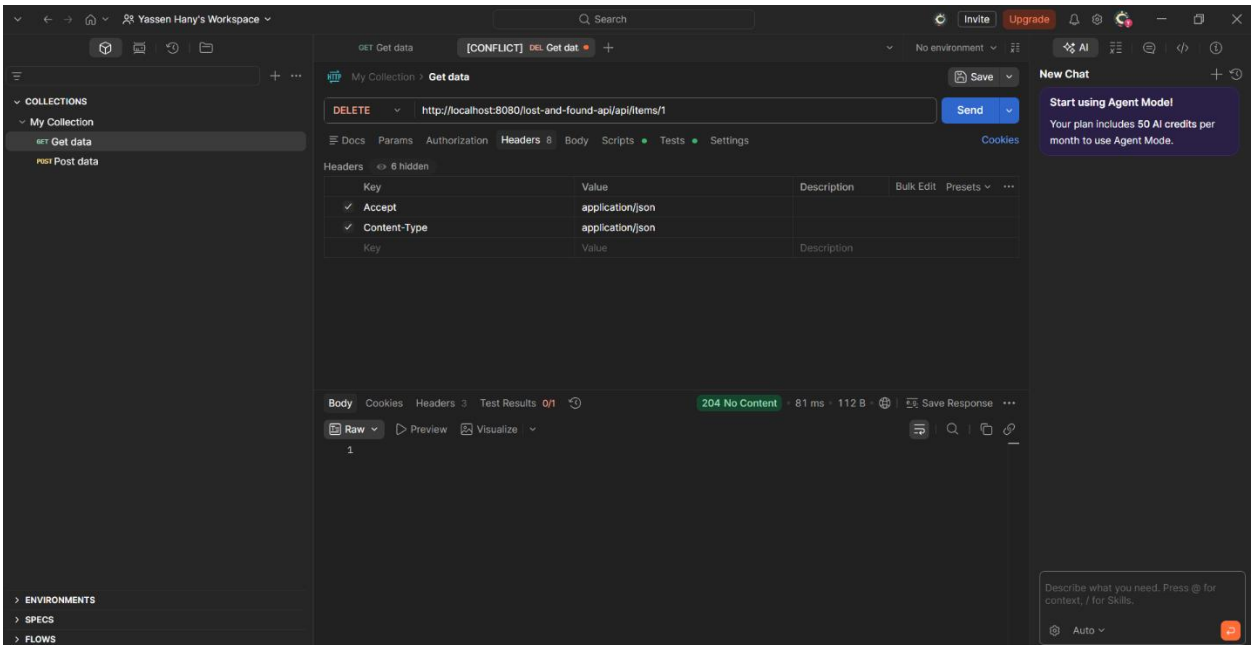
Endpoint: PUT /api/items/{id} & PUT /api/claims/{id}

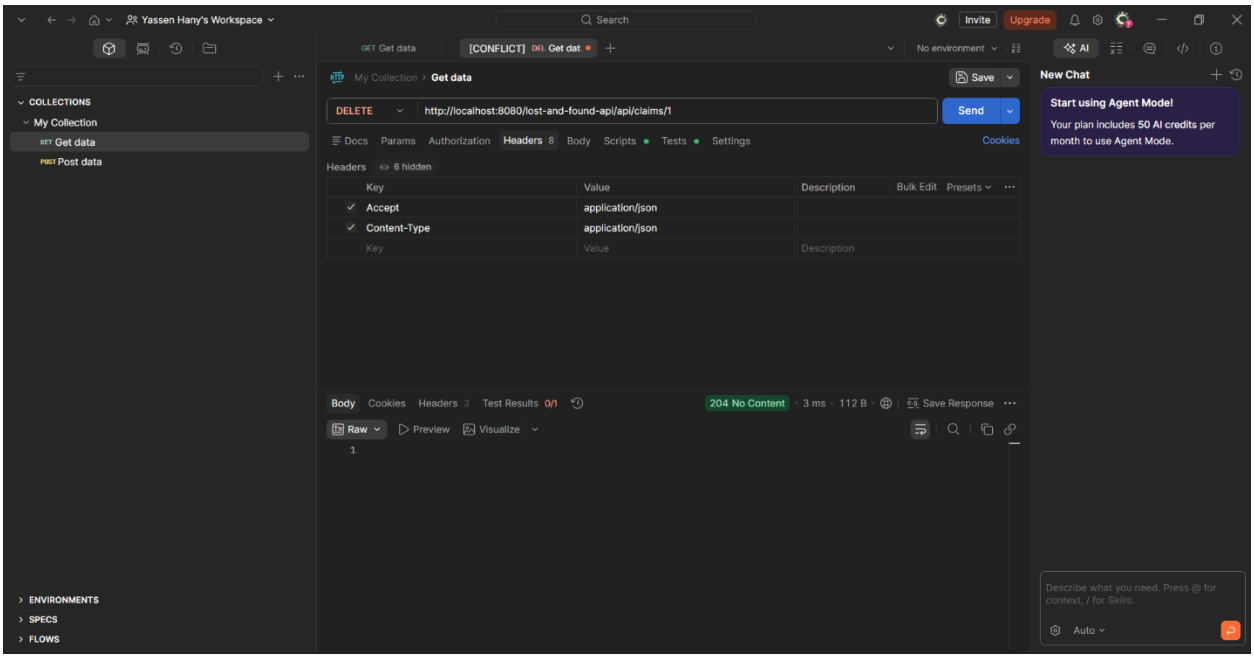




4. DELETE APIs (Remove Data)

Endpoint: DELETE /api/items/{id} & DELETE /api/claims/{id}





Part 3: Service Composition

Description of Composite Service

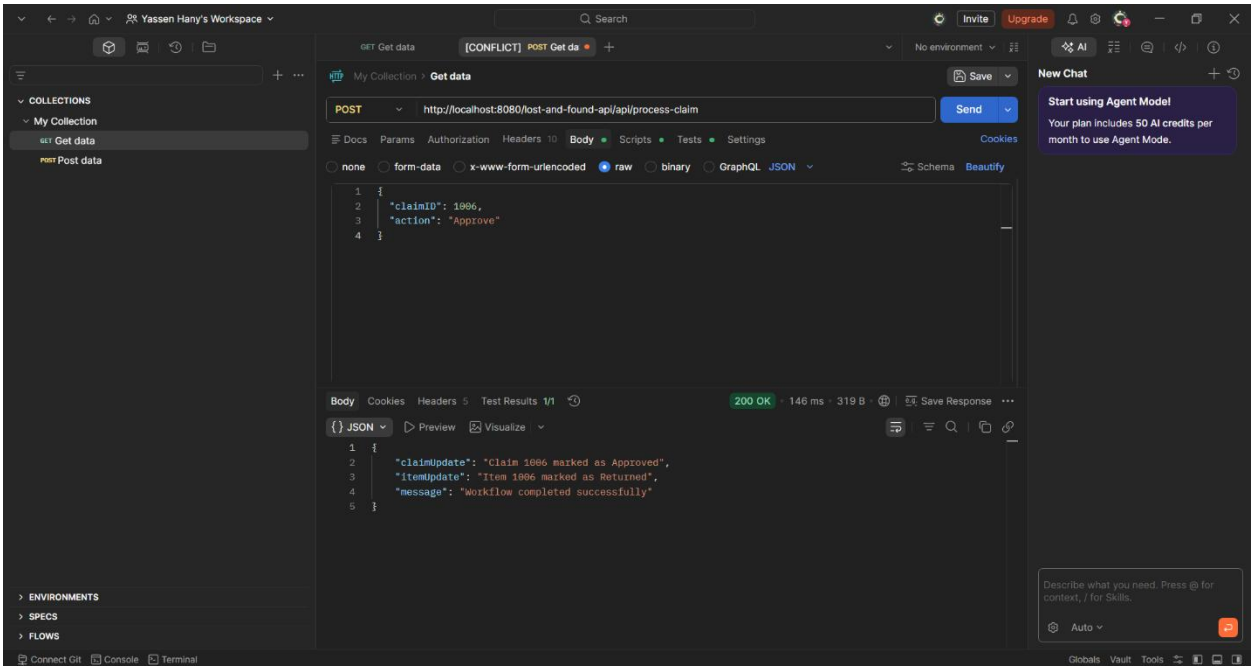
To represent a complete business workflow, a Composite Service was implemented at the endpoint /api/process-claim.

Workflow Explanation

When a security officer reviews a claim, updating the claim status alone is insufficient; the system must also secure the physical item. This composite service orchestrates two separate APIs in a single transaction:

1. **API Action 1:** Updates the Claim database record to "Approved" or "Rejected".
2. **API Action 2:** Locates the corresponding Item record and automatically updates its status to "Returned" or "Found".

Composite Service Test



Part 4: API Documentation

The following is the OpenAPI Specification documenting the RESTful services